

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

# GEAVANCEERDE COMPUTERARCHITECTUUR

Lab 3

**Thibaut Beck**
**Wannes Paesschesoone**

Academic year 2025–2026

# Inhoudsopgave

**1 Inleiding** **2**

**2 Theorie & Concepten** **2**
2.1 Warps . . . 2
2.2 Thread Divergence . . . 2
2.3 Striding . . . 2

**3 Oefeningen** **3**
3.1 Deel 1 - warps . . . 3
3.1.1 Kernels Code . . . 3
3.1.2 Resultaten en analyse . . . 4
3.2 Deel 2 - Thread Divergence . . . 5
3.2.1 Kernels Code . . . 5
3.2.2 Resultaten en analyse . . . 6

**4 Conclusie** **7**

**5 Appendix** **7**

# 1 Inleiding

In dit CUDA-labo werd onderzocht wat warps zijn en hoe striding en thread divergence de uitvoering beïnvloeden. Om dit te illustreren, werden twee programma's ontwikkeld. Het eerste programma inverteert de kleuren van een RGB-afbeelding, waarbij elke kleur 8 bits gebruikt (waarden 0–255). Hierbij werd onderzocht hoe volle versus gedeeltelijke warps de rekentijd beïnvloeden.

Het tweede programma richt zich op thread divergence. Hiervoor werden twee kernels geschreven: één met interleaved-RGB en één met planar indeling (RRRGGGBBB). In beide gevallen werd een inversiekernel toegepast, maar voor de rode kleur werd bovendien een 1D Gaussian blur berekend over de naburige pixels, terwijl de andere kleuren zoals in het eerste programma werden geïnverteerd.

# 2 Theorie & Concepten

## 2.1 Warps

Een warp is een groep van 32 threads die samen op een SM draaien. Volle warps benutten alle cores optimaal, gedeeltelijke warps laten sommige cores idle, wat de rekentijd verhoogt.
**Voorbeeld:**

```
foo<<<2,64>>>();  // 2 blocks, 64 threads -> 4 warps
foo<<<4,33>>>();  // 4 blocks, 33 threads -> 8 warps
```

## 2.2 Thread Divergence

Thread divergence treedt op wanneer threads binnen een warp verschillende instructies uitvoeren. Divergente paden worden sequentieel uitgevoerd, waardoor sommige cores tijdelijk idle zijn. Hierbij heeft één van de paden een hogere computatie complexiteit dan het andere pad.
**Voorbeeld:**

```
if (tid % 2 == 0) c[tid] = a[tid] + b[tid];
else c[tid] = a[tid] * b[tid];
```

## 2.3 Striding

Wordt gebruikt als er minder threads zijn dan elementen. Elke thread verwerkt meerdere elementen.
**Voorbeeld:** 256 threads verwerken 260 elementen, sommige threads rekenen twee elementen in plaats van één. Zo is het niet nodig om een volledige extra warp op te starten voor de resterende 4 elementen.

# 3 Oefeningen

## 3.1 Deel 1 - warps

In dit deel werd onderzocht hoe het aantal threads per block en de grootte van warps de rekentijd beïnvloeden bij het inverteren van een RGB-afbeelding. Daarnaast werd ook de invloed van stringding onderzocht. Hiervoor werden twee kernels gebruikt:

### 3.1.1 Kernels Code

**Kernel zonder striding:**

- Elke thread verwerkt exact één element van de 1D-geflattened image array.
- Als er meer threads zijn dan elementen, blijven sommige threads idle.

```
__global__ void invert_image(uint8_t* input_img, uint8_t* output_img, int total_size){
    int tid = threadIdx.x + blockDim.x * blockIdx.x;

    if (tid < total_size){
        output_img[tid] = 255 - input_img[tid];
    }
}
```

Listing 1: Inverteren foto zonder striding

**Kernel met striding:**

- Hier verwerkt elke thread meerdere elementen volgens de stride.
- Zo kan een kleiner aantal threads toch de volledige afbeelding verwerken.
- Dit voorkomt dat gedeeltelijke warps een volledige extra warp moeten oproepen voor een klein aantal resterende pixels.

```
__global__ void invert_image_stride(uint8_t* input_img, uint8_t* output_img, int total_size){
    int index = threadIdx.x + blockDim.x * blockIdx.x;

    int stride = blockDim.x * gridDim.x;

    for (int i = index; i < total_size; i += stride) {
        output_img[i] = 255 - input_img[i];
    }
}
```

Listing 2: Inverteren foto met striding

### 3.1.2 Resultaten en analyse

De performance werd gemeten voor verschillende aantallen threads per block, zowel volle als gedeeltelijke warps met en zonder striding.

| Threads/Block | Blocks/Grid | Avg. Time (ms) |
|---|---|---|
| **Full Warps** | | |
| 32 | 115200 | 0.226768 |
| 64 | 57600 | 0.091444 |
| 128 | 28800 | 0.056243 |
| 256 | 14400 | 0.056310 |
| 512 | 7200 | 0.059719 |
| 1024 | 3600 | 0.089459 |
| **Partial Warps** | | |
| 16 | 230400 | 0.431169 |
| 20 | 184320 | 0.348114 |
| 100 | 36864 | 0.070232 |
| 200 | 18432 | 0.068552 |
| 400 | 9216 | 0.071997 |
| **Full Warps (Striding)** | | |
| 32 | 7200 | 0.105032 |
| 64 | 3600 | 0.059209 |
| 128 | 1800 | 0.044112 |
| 256 | 900 | 0.044270 |
| 512 | 450 | 0.044836 |
| 1024 | 225 | 0.065010 |
| **Partial Warps (Striding)** | | |
| 16 | 14400 | 0.196002 |
| 20 | 11520 | 0.161160 |
| 100 | 2304 | 0.055863 |
| 200 | 1152 | 0.053783 |
| 400 | 576 | 0.058011 |

Tabel 1: Gemiddelde uitvoeringstijden van de twee kernels voor verschillende threads per block

- **Full warps:**
  - De rekentijd daalt snel wanneer threads per block toenemen tot 128 of 256.
  - Boven 256 threads neemt de tijd iets toe, omdat er minder warps nodig zijn en sommige SMs minder goed gevuld worden.
- **Partial warps:**
  - Threads die geen volle warp vormen (bv. 16 of 20) zijn duidelijk trager, tot bijna het dubbele van de tijd van een volle warp.
  - Dit toont het verlies aan efficiëntie door idle cores bij gedeeltelijke warps.
- **Striding:**
  - Striding vermindert de negatieve impact van gedeeltelijke warps: de rekentijd daalt aanzienlijk.
  - Bijvoorbeeld 16 threads met striding verwerkt de volledige afbeelding bijna twee keer zo snel als zonder striding.
  - Dit laat zien dat striding de GPU beter benut door threads meerdere elementen te laten verwerken.

**Conclusie:** Volle warps benutten de GPU het efficiëntst, terwijl gedeeltelijke warps leiden tot idle cores en langere rekentijden. Striding vermindert dit probleem door threads meerdere elementen te laten verwerken, waardoor de uitvoeringstijd stabiel blijft ongeacht het aantal threads per block.

## 3.2 Deel 2 - Thread Divergence

In dit deel werd onderzocht hoe thread divergence de uitvoeringstijd beïnvloedt bij het verwerken van een RGB-afbeelding. Twee kernels werden gebruikt: één voor interleaved RGB-indeling en één voor planar indeling (zie appendix listing 5).

### 3.2.1 Kernels Code

**Interleaved RGB kernel:**

- Threads binnen een warp volgen verschillende paden: sommige berekenen een 1D Gaussian blur op de R-component, anderen inverteren G en B.
- Dit veroorzaakt thread divergence: sommige cores zijn idle terwijl andere nog werken. Dit komt voor doordat de Gaussian blur een aanzienlijk complexere berekening is dan het inverteren van een kleur.

```
__global__ void convert_image(uint8_t* input_img, uint8_t* output_img, int total_size){
    int tid = threadIdx.x + blockDim.x * blockIdx.x;

    if (tid >= 6 && tid < total_size - 6 && tid % 3 == 0) {
        output_img[tid] = 0.1 * input_img[tid - 6] +
                          0.25 * input_img[tid - 3] +
                          0.5 * input_img[tid] +
                          0.25 * input_img[tid + 3] +
                          0.1 * input_img[tid + 6];
    } else if (tid < total_size) {
        output_img[tid] = 255 - input_img[tid];
    }
}
```

Listing 3: Gaussian blur en invert kernel (RGB)

**Planar kernel:**

- Afbeelding wordt eerst omgezet naar planar indeling (`RR...RGG...GBB...B`).
- Threads binnen een warp voeren homogeen dezelfde berekening uit, wat thread divergence vermindert.
- R-component krijgt Gaussian blur, G en B worden geïnverteerd.

```
__global__ void convert_image_planar(uint8_t* input_img, uint8_t* output_img, int num_pixels)
    {
    int tid = threadIdx.x + blockDim.x * blockIdx.x;
    int total_elements = num_pixels * 3;

    if (tid >= total_elements) return;

    if (tid < num_pixels) {
        if (tid >= 2 && tid < num_pixels - 2) {
            output_img[tid] = 0.1 * input_img[tid - 2] + 0.25 * input_img[tid - 1] +
                              0.5 * input_img[tid] + 0.25 * input_img[tid + 1] +
                              0.1 * input_img[tid + 2];
        } else {
            output_img[tid] = input_img[tid];
        }
    } else {
        output_img[tid] = 255 - input_img[tid];
    }
}
```

Listing 4: Gaussian blur en invert kernel (Planar)

### 3.2.2 Resultaten en analyse

De performance werd gemeten voor verschillende aantallen threads per block, zowel volle als gedeeltelijke warps, en voor de standaard RGB-indeling als de planar indeling.

| Threads/Block | Blocks/Grid | Avg. Time (ms) |
|---|---|---|
| **Full Warps (RGB)** | | |
| 32 | 115200 | 0.521131 |
| 64 | 57600 | 0.498990 |
| 128 | 28800 | 0.499194 |
| 256 | 14400 | 0.492685 |
| 512 | 7200 | 0.423275 |
| 1024 | 3600 | 0.461463 |
| **Partial Warps (RGB)** | | |
| 16 | 230400 | 0.799954 |
| 20 | 184320 | 0.641707 |
| 100 | 36864 | 0.514562 |
| 200 | 18432 | 0.453708 |
| 400 | 9216 | 0.424479 |
| **Full Warps (Planar)** | | |
| 32 | 115200 | 0.257191 |
| 64 | 57600 | 0.180588 |
| 128 | 28800 | 0.166926 |
| 256 | 14400 | 0.169089 |
| 512 | 7200 | 0.171960 |
| 1024 | 3600 | 0.209132 |
| **Partial Warps (Planar)** | | |
| 16 | 230400 | 0.488114 |
| 20 | 184320 | 0.387317 |
| 100 | 36864 | 0.207713 |
| 200 | 18432 | 0.191628 |
| 400 | 9216 | 0.185118 |

Tabel 2: Gemiddelde uitvoeringstijden van de divergentie-kernels voor verschillende threads per block

- **RGB kernel:**
  - Gedeeltelijke warps (bv. 16 of 20 threads) zijn trager door idle cores, de warp wordt niet volledig benut.
  - Volle warps benutten de GPU efficiënter, maar sommige heterogene berekeningen (bv. blur versus invert) veroorzaken nog steeds korte periodes waarin threads moeten wachten. Dit fenomeen, waarbij threads wachten op andere threads die complexere berekeningen uitvoeren, noemen we **thread divergence**.
- **Planar kernel:**
  - Homogene berekeningen binnen een warp verminderen thread divergence.
  - Dit resulteert in kortere rekentijden, vooral bij lagere thread counts.
- **Volle vs gedeeltelijke warps:**
  - Net als bij deel 1 benutten volle warps de GPU beter.
  - Gedeeltelijke warps verliezen efficiëntie door idle cores.

**Conclusie:** Thread divergence kan de uitvoeringstijd significant verhogen. Het gebruik van planar indeling helpt om divergence te verminderen en de GPU beter te benutten.

# 4 Conclusie

In dit labo werd onderzocht hoe warps, striding en thread divergence de uitvoeringstijd van CUDA-kernels beïnvloeden. Enkele belangrijke bevindingen zijn:

- **Warps:** Volle warps (32 threads) benutten de GPU het efficiëntst. Gedeeltelijke warps leiden tot idle cores en hogere rekentijden. Het is belangrijk om threads per block zo te kiezen dat zoveel mogelijk volle warps ontstaan.
- **Striding:** Wanneer het aantal threads kleiner is dan het aantal te verwerken elementen, kan striding gebruikt worden. Elke thread verwerkt dan meerdere elementen, waardoor de GPU beter wordt benut en de uitvoeringstijd stabiel blijft, ongeacht het aantal threads per block. Striding voorkomt dat een volledige extra warp moet worden opgestart voor een klein aantal resterende elementen.
- **Thread Divergence:** Divergentie treedt op wanneer threads binnen een warp verschillende berekeningen uitvoeren, zoals bij een combinatie van invert en Gaussian blur. Dit veroorzaakt korte periodes waarin sommige cores idle zijn. Het gebruik van een planar indeling vermindert divergence door homogeen berekeningen binnen een warp uit te voeren.
- **Praktische implicaties:** Voor optimale prestaties is het belangrijk een balans te vinden tussen het aantal threads per block, het gebruik van striding, en het minimaliseren van thread divergence. Planar data-indelingen en het kiezen van volle warps dragen significant bij aan een hogere efficiëntie.

Samengevat laten de experimenten zien dat de GPU-prestaties sterk afhangen van de warp-indeling en thread-structuur. Door deze factoren bewust te ontwerpen, kunnen rekentijden aanzienlijk worden verminderd en de beschikbare cores optimaal worden benut.

# 5 Appendix

```
size_t img_size_bytes = total_elements * sizeof(uint8_t);

uint8_t* planar_format = (uint8_t*)malloc(img_size_bytes);

for (int i = 0; i < num_pixels; i++){
    planar_format[i] = image_array[i*3];
    planar_format[i + num_pixels] = image_array[i*3+1];
    planar_format[i + 2*num_pixels] = image_array[i*3+2];
}
```

Listing 5: Interleaved naar planar RGB